Trabajo Práctico 3 – Memoria

SISTEMAS OPERATIVOS

Índice

1. ELF y el proceso de enlace **3**
1.1. Del código fuente al ejecutable 4
1.2. El programa que vamos a observar: `memoria.c` 5

2. ELF: cabecera, secciones y símbolos **6**
2.1. De los objetos fuente a las secciones 7

3. Segmentos y carga en memoria **8**
3.1. Seguir un símbolo externo 8
3.2. Páginas, memoria virtual e intercambio 9

4. `/proc/<pid>/maps`: del segmento a la región **9**
4.1. Observar el cargador con `gdb` 10

Este TP sigue el paso de un programa desde el código fuente hasta un proceso en ejecución. Primero construirás `hello.c` para observar compilación, enlace y formatos de archivo. Luego crearás `memoria.c` para relacionar sus símbolos, secciones y segmentos con las regiones que el núcleo (kernel) carga en memoria.

1. ELF y el proceso de enlace

ELF (*Executable and Linkable Format*) organiza código, datos, símbolos, reubicaciones y bibliotecas para que el enlazador y el núcleo puedan usar un objeto. Para ubicar los tres tipos principales en la ruta del programa, sigamos el caso de `hello.c`:

1. **Archivo reubicable (*relocatable file*)**. Contiene código y datos todavía enlazables con otros objetos. Es la salida de compilar con `gcc -c hello.c`; en Linux suele tener extensión `.o` (y `.ko` en un módulo del núcleo), y todavía no está listo para iniciar.
2. **Archivo ejecutable (*executable file*)**. Es el resultado usual de enlazar objetos y bibliotecas; queda listo para iniciar. Un script de shell no es un ELF: quien se ejecuta es su intérprete. En Linux suele no llevar extensión, como el ejecutable `hello`.
3. **Objeto compartido (*shared object*)**. Es una biblioteca que puede enlazarse o cargarse junto con un programa; en Linux suele tener extensión `.so`. No es un paso obligatorio después del ejecutable, sino una rama paralela que puede participar en el enlace o en la carga.

El recorrido principal de este práctico es `hello.c` ⟶ `hello.o` ⟶ `hello` ⟶ proceso.

Creá `hello.c` con este contenido:

```c
#include <stdio.h>

int main() {
    printf("hello\n");
    return 0;
}
```

```
$ gcc -c hello.c -o hello.o
$ gcc hello.c -o hello
$ file hello hello.o
hello: ELF 64-bit LSB pie executable, x86-64, dynamically linked,
    ...
hello.o: ELF 64-bit LSB relocatable, x86-64, ...
```

1.1. Del código fuente al ejecutable

Al compilar, el comando `gcc` coordina internamente cuatro etapas: el preprocesador (*preprocessor*) expande `#include`, macros y condicionales; el compilador (*compiler*) traduce el código a ensamblador; el ensamblador (*assembler*) produce un archivo objeto `.o`; y el enlazador (*linker*) combina objetos y bibliotecas para formar el ejecutable. Una invocación normal ejecuta todas las etapas y entrega el ejecutable; las opciones siguientes permiten detenerse y conservar resultados intermedios. Así, `gcc` coordina el recorrido completo: no hay que invocar manualmente cada herramienta para compilar un programa habitual:

```
$ gcc -E hello.c -o hello.i
$ gcc -S hello.c -o hello.s
$ gcc -c hello.c -o hello.o
$ gcc hello.o -o hello
```

Las extensiones `.i`, `.s` y `.o` identifican, respectivamente, resultados preprocesados, ensamblador y archivos objeto. El último comando vuelve a enlazar el objeto y produce el ejecutable.

Una biblioteca estática contiene objetos que se copian al ejecutable; una compartida queda como dependencia y sus símbolos se resuelven al cargar. Para comparar ambas variantes usaremos una biblioteca mínima: `saludo.c` define la función `saludar()` y `main.c` la invoca.

```c
#include <stdio.h>

void saludar() {
    printf("Hola desde libsaludo\n");
}
```

```c
void saludar();

int main() {
    saludar();
    return 0;
}
```

Primero compilá los dos archivos fuente como objetos. Este paso es común a las dos variantes:

```
$ gcc -Wall -Wextra -c saludo.c -o saludo.o
$ gcc -Wall -Wextra -c main.c -o main.o
```

Enlace estático. Un archivo `.a` es normalmente una biblioteca estática (un archivo *archive*) que agrupa objetos. El prefijo `lib` y el sufijo `.a` permiten que `-lsaludo` encuentre `libsaludo.a`; `-L.` agrega el directorio actual a la búsqueda. Creá la biblioteca y enlazala con el programa:

```
$ ar rcs libsaludo.a saludo.o
$ gcc main.o -L. -lsaludo -o saludo-estatico
$ file libsaludo.a saludo-estatico
```

Enlace dinámico. Creá un objeto compartido (`.so`) y enlazalo dejando la biblioteca como dependencia del ejecutable:

```
$ gcc -Wall -Wextra -fPIC -shared saludo.c -o libsaludo.so
$ gcc -Wall -Wextra main.c -L. -lsaludo -Wl,-rpath,'$ORIGIN' -o
    saludo-compartido
$ file libsaludo.so saludo-compartido
$ ldd saludo-compartido
$ readelf -d saludo-compartido
```

En la variante estática, la implementación de `saludar` queda dentro del ejecutable y no hace falta conservar `libsaludo.a` para ejecutarlo. En la variante compartida, el ejecutable conserva una dependencia de `libsaludo.so`; por eso `ldd` y `readelf -d` permiten verla. Si cambia una biblioteca estática hay que volver a enlazar; una biblioteca compartida puede reemplazarse sin recompilar el programa, siempre que se conserve una interfaz compatible.

EXTRA

Compará las variantes con `file`, `ldd` y `readelf -d`: registrá tamaño, dependencias e intérprete. Estos resultados pueden cambiar entre arquitecturas y distribuciones.

1.2. El programa que vamos a observar: `memoria.c`

Creá `memoria.c`. Este único programa contiene los objetos que vamos a seguir: datos inicializados y no inicializados, una cadena de sólo lectura, una función, una variable local y una reserva en el heap. También imprime sus direcciones y su identificador de proceso (PID, *Process ID*), y permanece vivo para poder observar su mapa.

```c
#include <inttypes.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_data = 4;
int global_data_2;
static const char message[] = "hello, world";

static int marker() {
    return 7;
}

int main() {
    int local_data = 3;
    int *heap_data = malloc(sizeof *heap_data);

    if (heap_data == NULL) {
        perror("malloc");
        return 1;
    }

    *heap_data = 42;
    printf("PID: %d\n", getpid());
    printf("mensaje: %s @ %p\n", message, (void *)message);
    printf("global_data: %d @ %p\n", global_data, (void *)&global_data);
    printf("global_data_2: %d @ %p\n", global_data_2, (void *)&
        global_data_2);
    printf("local_data: %d @ %p\n", local_data, (void *)&local_data);
    printf("funcion: 0x%" PRIxPTR "\n", (uintptr_t)marker);
    printf("heap_data: %p\n", (void *)heap_data);
    fflush(stdout);

    sleep(30);
    free(heap_data);
    return 0;
}
```

```
$ gcc -Wall -Wextra -O0 -g -o memoria memoria.c
$ ./memoria
```

No copies las direcciones de una ejecución: cambian con ASLR (Address Space Layout Randomization). En las próximas secciones vamos a comprobar, usando las herramientas del sistema, qué parte del archivo y qué región del proceso corresponde a cada objeto.

2. ELF: cabecera, secciones y símbolos

El comando `file` ofrece una clasificación rápida. Para inspeccionar los campos con más detalle usaremos `readelf`, una herramienta de `binutils`

que interpreta las estructuras ELF. Ahora inspeccionamos el archivo que acabamos de construir, sin cambiar todavía al proceso en ejecución:

```
$ file memoria
$ readelf -h memoria
$ readelf -h memoria | grep -E 'Class|Data|Type|Machine|Entry'
```

La cabecera informa el número mágico (`7f 45 4c 46`) y la clase del objeto (`ELF32` o `ELF64`). También indica el orden de bytes, la arquitectura, el tipo, el punto de entrada y la ubicación de las tablas. Con un ejecutable independiente de posición (PIE, *Position-Independent Executable*), `Type` suele ser `DYN`. Sin PIE puede ser `EXEC`; ambos siguen siendo ejecutables. La salida exacta depende de la arquitectura y de las opciones de compilación.

Usá la salida de tu propio `memoria`: no copies direcciones de una salida de referencia. Explicá qué informa cada campo y registrá sus valores.

2.1. De los objetos fuente a las secciones

Las secciones son la vista del compilador y del enlazador. Listalas y buscá los objetos del programa:

```
$ readelf -SW memoria
$ readelf -sW memoria | grep -E 'main|marker|message|global_data|
    printf'
$ nm -C memoria | grep -E 'main|marker|message|global_data'
$ objdump -d -j .text memoria
$ objdump -s -j .data memoria
$ objdump -s -j .rodata memoria
```

- `.text` contiene `main` y `marker`; `.rodata` contiene `message` y otras cadenas.
- `.data` contiene `global_data`; `.bss` (BSS, *Block Started by Symbol*) contiene `global_data_2`. El archivo guarda el tamaño de `.bss`, no todos sus ceros.
- `.symtab` contiene símbolos de enlace y depuración, y `.dynsym`, símbolos del enlace dinámico. Un ejecutable *stripped* puede perder la primera.
- `local_data` y `heap_data` no son secciones ELF: aparecen en la pila y el heap del proceso cuando el programa se ejecuta.

En `readelf -sW`, `Value` es una dirección o valor relativo y `Ndx` identifica la sección. `printf` suele figurar como `UND`, porque su implementación vive en una biblioteca compartida. La relación entre ese símbolo externo, `.plt` y `.got.plt` se retomará más adelante.

EXTRA

Copiá `memoria` como `memoria-stripped`, ejecutá `strip` y repetí `nm -C` y `readelf -sW`. Explicá qué información se pierde y por qué el programa

todavía puede ejecutarse.

3. Segmentos y carga en memoria

Las secciones organizan el archivo para el compilador y el enlazador; los segmentos describen lo que el cargador proyecta en memoria. Inspeccioná ambos vínculos:

```
$ readelf -lW memoria
$ readelf -SW memoria
```

La tabla de cabecera de programa (PHT, *Program Header Table*) relaciona las secciones con los segmentos que el cargador puede proyectar. Cada segmento `LOAD` describe una región y sus banderas principales son `R` (lectura), `W` (escritura) y `E` (ejecución). Normalmente `.text` y `.rodata` van en un segmento `R E`; `.data` y `.bss`, en uno `R W`.

`FileSiz` indica cuántos bytes del segmento ocupan espacio en el archivo; `MemSiz` indica cuántos bytes necesita en memoria. Para `.bss`, el cargador completa con ceros la diferencia entre ambos, por eso el segmento de datos puede tener `MemSiz` mayor que `FileSiz`. Un segmento puede contener varias secciones. El núcleo redondea sus límites a páginas, habitualmente de 4 KiB, y conserva sus protecciones.

3.1. Seguir un símbolo externo

Una llamada a `printf` suele pasar por la PLT (*Procedure Linkage Table*) y la GOT (*Global Offset Table*): el código salta a la PLT, ésta usa una entrada de la GOT, el enlazador dinámico resuelve el símbolo y guarda la dirección para las llamadas siguientes. En la primera llamada, la entrada de la GOT puede apuntar nuevamente a la PLT; el enlazador dinámico resuelve la dirección y actualiza esa entrada. En las siguientes llamadas se usa la dirección ya resuelta.

```
$ objdump -d -j .plt memoria
$ readelf -rW memoria
$ readelf -x .got.plt memoria
$ LD_BIND_NOW=1 ./memoria
$ LD_DEBUG=bindings ./memoria 2>&1 | less
```

Las instrucciones concretas dependen de la arquitectura. El enlace perezoso es una estrategia, no una garantía idéntica entre versiones de PIE y binutils. Esta vista queda como orientación; el camino principal continúa con la carga del mismo archivo.

3.2. Páginas, memoria virtual e intercambio

La memoria virtual da a cada proceso un espacio propio. El núcleo traduce direcciones a páginas físicas y las carga bajo demanda: *page-in* trae una página desde un archivo o intercambio (*swap*); *page-out* la escribe para liberar memoria. `vmstat` resume memoria, intercambio, E/S, sistema y CPU:

```
$ vmstat 1 5
$ free -h
$ cat /proc/meminfo
$ cat /proc/swaps
```

`free` muestra memoria libre; `si`, páginas llevadas a RAM; y `so`, páginas enviadas a swap por segundo. Un `so` aislado no diagnostica un problema: correlacioná con `free`, `top` y `/proc/meminfo`.

EXTRA

Investigá memoria anónima, páginas respaldadas por archivos y swap en `man 5 proc`, especialmente `meminfo`, `status` y `smaps`. No uses `swapoff` en una máquina compartida.

4. /proc/<pid>/maps: del segmento a la región

El sistema de archivos virtual procfs (*proc filesystem*) expone el estado del sistema y de los procesos. La información de un proceso con PID `1234` está en `/proc/1234/`; `/proc/self/` refiere al proceso que consulta ese archivo.

Ahora ejecutá el mismo `memoria` y compará sus regiones con los segmentos de `readelf -lW`:

```
$ ./memoria &
$ pid=$!
$ cat /proc/$pid/maps
$ cat /proc/$pid/status | grep -E 'Vm|Threads'
$ wait $pid
```

Una línea de `maps` tiene esta forma:

```
55a1...-55a2... r-xp 00002000 08:01 12345 /home/usuario/memoria
7f10...-7f12... r-xp 00000000 08:01 67890 /usr/lib/libc.so.6
7ffd...-7fff... rw-p 00000000 00:00 0 [stack]
```

Los campos son: rango virtual (inicio inclusivo, fin exclusivo), permisos `rwx`, tipo de compartimiento `p`/`s`, desplazamiento dentro del archivo respaldante, dispositivo, i-nodo y ruta. Los ceros y los nombres como `[heap]` o `[stack]` indican regiones anónimas. El ejecutable puede aparecer en varias líneas porque sus segmentos tienen protecciones distintas; también aparecen el enlazador dinámico, `libc`, el heap y la pila.

4.1. Observar el cargador con `gdb`

`gdb` (*GNU Debugger*) es un depurador interactivo: permite iniciar un programa, detenerlo en un punto elegido e inspeccionar su estado. Al detenernos en `main`, el cargador ya preparó el ejecutable y sus bibliotecas; por eso podemos observar el proceso mientras está detenido. `break main` fija el punto de detención, `run` inicia el programa, `backtrace` muestra la pila de llamadas, `info proc mappings` muestra sus regiones de memoria, `next` avanza una línea y `continue` reanuda la ejecución.

```
$ gdb ./memoria
(gdb) break main
(gdb) run
(gdb) backtrace
(gdb) print &global_data
(gdb) info proc mappings
(gdb) next
(gdb) continue
(gdb) quit
```

También podés usar `pmap -x <pid>` como vista alternativa. Para inspeccionar un ELF, `readelf`, `objdump` y `nm` ofrecen vistas complementarias de sus cabeceras, secciones y símbolos.